

Animasi Kubus 3D Di Terminal Menggunakan C by Ilkomerz Timothee Chalamet

Fase 1: Fondasi Matematis

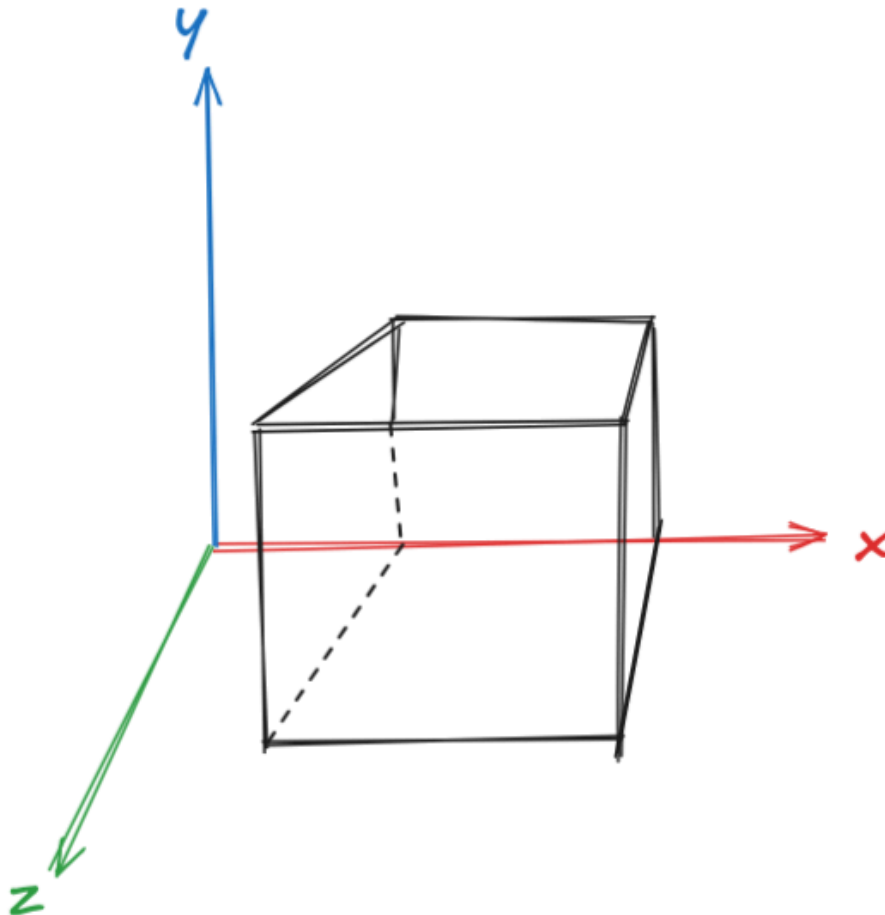
Sebelum menggambar, kita akan memahami cara merepresentasi dan memanipulasi ruang 3 dimensi.

1. Mendefinisikan Sistem Koordinat

Bayangkan sebuah grid 3D yang dimana:

- Sumbu X: Bergerak ke kiri dan ke kanan
- Sumbu Y: Bergerak ke atas dan ke bawah
- Sumbu Z: Bergerak ke depan dan ke belakang

Daripada hanya membuat 8 sudut sebuah kubus, kita akan membuat kubus sebagai koleksi titik sepanjang keenam permukaannya.



2. Matrix Rotasi

Untuk membuat kubusnya berputar, kita akan merotasikan setiap titik dalam ruang 3D sekeliling sumbu menggunakan trigonometri. Rotasi 3D akan di komputasi menggunakan matrix 3×3 .

Sumbu X

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix}$$

Sumbu Y

$$R_y(\theta) = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix}$$

Sumbu Z

$$R_z(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Penjabaran Seluruh Titik

$$(i \ j \ k) \times R_{x(A)} \times R_{y(B)} \times R_{z(C)}$$

$$= (i \ j \ k) \times \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(A) & -\sin(A) \\ 0 & \sin(A) & \cos(A) \end{pmatrix} \times \begin{pmatrix} \cos(B) & 0 & \sin(B) \\ 0 & 1 & 0 \\ -\sin(B) & 0 & \cos(B) \end{pmatrix} \times \begin{pmatrix} \cos(C) & -\sin(C) & 0 \\ \sin(C) & \cos(C) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$= (i \ j \cos(A) + k \sin(A) \quad -j \sin(A) + k \cos(A)) \times \begin{pmatrix} \cos(B) & 0 & \sin(B) \\ 0 & 1 & 0 \\ -\sin(B) & 0 & \cos(B) \end{pmatrix} \times \begin{pmatrix} \cos(C) & -\sin(C) & 0 \\ \sin(C) & \cos(C) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$= (i \cos(B) + j \sin(A) \sin(B) - k \cos(A) \sin(B) \quad j \cos(A) + k \sin(A) \quad i \sin(B) - j \sin(A) \cos(B) + k \cos(A) \cos(B))$$

$$\times \begin{pmatrix} \cos(C) & -\sin(C) & 0 \\ \sin(C) & \cos(C) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$= (i \cos(B) \cos(C) + j \sin(A) \sin(B) \cos(C) - k \cos(A) \sin(B) \cos(C)$$

$$+ j \cos(A) \sin(C) + k \sin(A) \sin(C)$$

$$- (i \cos(B) + j \sin(A) \sin(B) - k \cos(A) \sin(B)) \sin(C)$$

$$+ j \cos(A) \cos(C) + k \sin(A) \cos(C)$$

$$i \sin(B) - j \sin(A) \cos(B) + k \cos(A) \cos(B))$$

Dimana:

$$X = i \cos(B) \cos(C) + j \sin(A) \sin(B) \cos(C) - k \cos(A) \sin(B) \cos(C) \\ + j \cos(A) \sin(C) + k \sin(A) \sin(C)$$

$$Y = -(i \cos(B) + j \sin(A) \sin(B) - k \cos(A) \sin(B)) \sin(C) \\ + j \cos(A) \cos(C) + k \sin(A) \cos(C)$$

$$Z = i \sin(B) - j \sin(A) \cos(B) + k \cos(A) \cos(B)$$

Kode C Untuk Fungsi Rotasi 3D

```
#include <math.h>

#define PI 3.14159265358979323846

// Lakukan prekalkulasi untuk efisiensi
double DEG2RAD(double x) {
    return ((x) * PI / 180.0);
}
double A, B, C;
double sinA, sinB, sinC;
double cosA, cosB, cosC;

void inis(){
    double radA = DEG2RAD(A);
    double radB = DEG2RAD(B);
    double radC = DEG2RAD(C);

    sinA = sin(radA);
    sinB = sin(radB);
    sinC = sin(radC);

    cosA = cos(radA);
    cosB = cos(radB);
    cosC = cos(radC);
}

double sumbuX(int i, int j, int k){
    return i * cosB * cosC + j * sinA * sinB * cosC + k * cosA * -sinB * cosC
        + j * cosA * sinC + k * sinA * sinC;
}

double sumbuY(int i, int j, int k){
    return i * cosB * -sinC + j * sinA * sinB * -sinC + k * cosA * sinB * sinC
        + j * cosA * cosC + k * sinA * cosC;
}

double sumbuZ(int i, int j, int k){
    return i * sinB + j * -sinA * cosB + k * cosA * cosB;
}
```

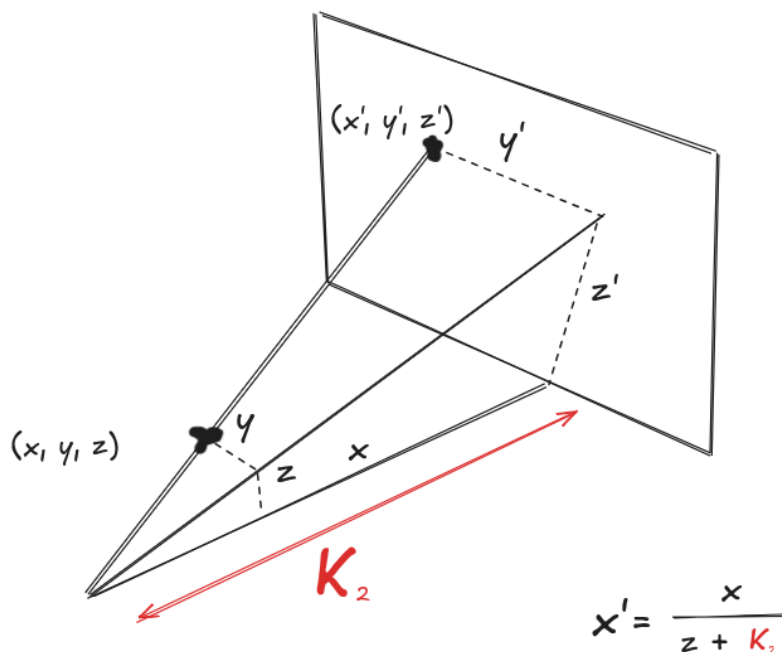
Fase 2: Proyeksi Objek 3D ke Terminal

Layar terminal itu datar (2D). Agar objek 3D terlihat realistis di layar yang datar, kita akan mengaplikasikan proyeksi perspektif.

- **Rumus Proyeksi:** Objek yang jauh akan terlihat kecil. Untuk mensimulasikan ini, kita membagi koordinat X dan Y yang sudah dirotasikan terhadap koordinat Z ditambah sebuah konstan yang merepresentasikan kejauhan kamera K_2 . Kita juga bisa mengalikannya dengan faktor skala K_1 agar kubusnya muat kedalam layar.

$$x_{\text{projected}} = \frac{X}{Z + K_2} \times K_1$$

$$y_{\text{projected}} = \frac{Y}{Z + K_2} \times K_1$$



- **Koreksi Aspect Ratio:** Karakter terminal biasanya dua kali lebih tinggi dibandingkan lebar. Ketika kita memetakan koordinatnya 1:1, kubus kita akan terlihat gepeng. Maka, proyeksi horizontal X harus dikalikan dengan *aspect ratio modifier* (biasanya 2) agar kubusnya berbentuk kotak sempurna.

```
// K1 mengontrol skala
// K2 mengontrol jarak kejauhan kamera
const int K1 = 40;
const int K2 = 100;

void titikPemrosesan(float cubeX, float cubeY, float cubeZ, char ch ){
    // Menghitung X, Y, dan Z yang dirotasikan
    float x = sumbuX(cubeX, cubeY, cubeZ);
    float y = sumbuY(cubeX, cubeY, cubeZ);
    float z = sumbuZ(cubeX, cubeY, cubeZ) + K2;
}
```

Fase 3: Memory dan Z-Buffering

Karena terminal merender dari kiri ke kanan, atas ke bawah, kita tidak bisa langsung menggambarkan titik selagi kita menghitungnya. Kita akan menyimpannya dalam *memory* terlebih dahulu.

- **Screen Buffer:** Buat array of character 1D. Ini akan merepresentasikan tiap sisi pada layar terminal.
- **Z-Buffer (Bufer Kedalaman):** Karena kubus memiliki sisi depan dan sisi belakang, kita hanya akan menggambar bagian yang paling dekat dengan kamera.
 - Daripada menyimpan nilai Z mentah, lebih cepat dikomputasi kalau kita menyimpan $1/Z$
 - Sebelum menulis karakter ASCII dalam *Screen Buffer* pada koordinat (x, y) , periksa *Z-Buffer* pada index yang sama.
 - Jika titik $1/Z$ lebih besar dari nilai yang tersimpan di *Z-Buffering*, artinya titik baru ini ada di depan. Timpa *Screen Buffer* dengan karakter baru dan perbarui *Z-Buffer* dengan $1/Z$

```
#define WIDTH 160
#define HEIGHT 44

char Buffer[WIDTH * HEIGHT];
float zBuffer[WIDTH * HEIGHT];
int bgASCII = ' ';

void titikPemrosesan(float cubeX, float cubeY, float cubeZ, char ch ){
    // Menghitung X, Y, dan Z yang dirotasikan
    float x = sumbuX(cubeX, cubeY, cubeZ);
    float y = sumbuY(cubeX, cubeY, cubeZ);
    float z = sumbuZ(cubeX, cubeY, cubeZ) + K2;

    // Semakin besar nilai 1/Z semakin dekat titiknya terhadap kamera.
    float seperZ = 1.0 / z;

    // Rumus Proyeksi
    int x_projected = (int)(x * seperZ * K1 * 2);
    int y_projected = (int)(y * seperZ * K1);

    // Geser koordinat (0,0) ke tengah WIDTH dan HEIGHT
    int screen_x = x_projected + (WIDTH / 2);
    int screen_y = y_projected + (HEIGHT / 2);

    //Periksa batasan dan Z-Buffering
    if (screen_x >= 0 && screen_x < WIDTH && screen_y >= 0 && screen_y < HEIGHT) {
        int idx = screen_y * WIDTH + screen_x;
        if (seperZ > zBuffer[idx]) {
            zBuffer[idx] = seperZ;
            Buffer[idx] = ch;
        }
    }
}
```

Fase 4: Animation Loop

Fungsi `main()` akan menjadi *infinite loop* yang terus menerus menghitung dan menggambar frame.

1. Persiapan Frame

Di awal *infinite loop*, kita akan membersihkan layar terminal untuk frame baru.

```
int main(){
    // Bersihkan layar terminal
    printf("\x1b[2J");

    // Infinite loop
    while(1){
        // Persiapkan fram
        // Reset screen buffer menjadi background ASCII (' ')
        memset(Buffer, bgASCII, WIDTH * HEIGHT);
        // Reset Z-Buffer
        memset(zBuffer, 0, WIDTH * HEIGHT * sizeof(float));

        // Panggil inis()
        inis();
    }
}
```

2. Menghasilkan Permukaan Kubus

Kita akan menghasilkan koordinat 3D untuk permukaan datar dari kubus. Anggaplah kubus kita berada di koordinat (0,0,0) dan sudutnya membentang dari -1 ke 1 pada semua sumbu.

```
const float STEP = 0.5; // kepadatan titik per permukaan

// Menghasilkan semua sisi kubus
for(float kubuX = -1; kubuX<1; kubuX += STEP){
    for(float kubuY = -1; kubuY<1; kubuY += STEP){

        // Sisi depan diisi dengan @
        titikPemrosesan(kubuX, kubuY, 1, '@');

        // Sisi belakang diisi dengan $
        titikPemrosesan(kubuX, kubuY, -1, '$');

        // Sisi kiri diisi dengan ~
        titikPemrosesan(-1, kubuY, kubuX, '~');

        // Sisi kanan diisi dengan #
        titikPemrosesan(1, kubuY, kubuX, '~');

        // Sisi atas diisi dengan ;
        titikPemrosesan(kubuX, -1, kubuY, ';');

        // Sisi bawah diisi dengan +
        titikPemrosesan(kubuX, 1, kubuY, '+');
```

3. Rendering

Setelah semua sisi diproses dan semua Buffer sudah dipenuhi character yang diinginkan:

```
// Kembalikan cursor terminal ke pojok kiri atas.
printf("\x1b[H");

// Loop Screen Buffer 1D dan print
for(int k=0; k < WIDTH * HEIGHT; k++){
    // Terminal tidak tahu kapan baris array selesai
    // jika index sudah melebihi ukuran terminal, print '\n'
    // Selainnya, print karakter yang kita simpan di buffer
    putchar(k % WIDTH ? Buffer[k] : '\n');
}
```

4. Langkah Animasi

```
// Increment sudutnya.
A += 0.6; // rotasi sumbu X
B += 0.3; // rotasi sumbu Y
C += 0.1; // rotasi sumbu Z

usleep(16000); // Kecepatan sekitar 60 fps
```

Fase 5: Dinamic Lighting

Kita akan mengubah kubus yang tadinya memiliki warna solid di setiap sisinya menjadi bereaksi terhadap cahaya.

1. Konsep Matematika

Agar sebuah titik pada kubus tahu seberapa terang ia terlihat, kita akan menghitung hubungan antara arah hadap permukaan tersebut dengan arah datangnya cahaya.

- **Vektor Normal (N):** Ini adalah vektor yang menunjuk tegak lurus keluar dari permukaan kubus. Misalnya, sisi depan memiliki normal $(0,0,1)$.
- **Vektor Cahaya (L):** Tentukan arah cahaya. Misal, cahaya datang dari arah depan-atas-kanan: $(0,1,-1)$.
 - $L_x = 0$
 - $L_y = 0$
 - $L_z = -1$
- **Dot Product:** Hasil perkalian titik antara vektor normal yang sudah dirotasi dengan vektor cahaya akan menghasilkan nilai **Luminasi**. Semakin sejajar normal dengan arah cahaya, semakin terang titik tersebut.

$$L = (N_x \cdot L_x) + (N_y \cdot L_y) + (N_z \cdot L_z)$$

- Jika hasilnya 1, maka permukaan berhadapan langsung dengan cahaya (terang)
- Jika 0, cahayanya merumpuk di permukaan (dalam bayangan)
- Jika kurang dari 0, permukaan berlawanan dari arah datang cahaya (hitam)

2. Pemetaan ASCII

Setelah mendapatkan dot product L , dan hasilnya lebih dari 0, kita akan memetakan nilai desimal ke dalam array of characters yang tersusun berdasarkan kepadatan visual.

- `const char brightness[] = ".,--~:;=!*#$@";`
- Kalikan L dengan panjang dari string untuk mendapatkan index array yang sesuai. L tinggi menghasilkan @, L yang rendah menghasilkan ..

```
void titikPemrosesan(float cubeX, float cubeY, float cubeZ, char ch ){
// Menghitung X, Y, dan Z yang dirotasikan
float x = sumbuX(cubeX, cubeY, cubeZ);
float y = sumbuY(cubeX, cubeY, cubeZ);
float z = sumbuZ(cubeX, cubeY, cubeZ) + K2;

// Rotasi Vektor Normal
float r_nx = sumbuX(nx, ny, nz);
float r_ny = sumbuY(nx, ny, nz);
float r_nz = sumbuZ(nx, ny, nz);

// Luminasi (Dot Product)
float L = (r_nx * 0) + (r_ny * 1) + (r_nz * -1);

// Semakin besar nilai 1/Z semakin dekat titiknya terhadap kamera.
float seperZ = 1.0 / z;

// Rumus Proyeksi
int x_projected = (int)(x * seperZ * K1 * 2);
int y_projected = (int)(y * seperZ * K1);

// Geser koordinat (0,0) ke tengah WIDTH dan HEIGHT
int screen_x = x_projected + (WIDTH / 2);
int screen_y = y_projected + (HEIGHT / 2);

//Periksa batar dan Z-Buffering
if (screen_x >= 0 && screen_x < WIDTH && screen_y >= 0 && screen_y < HEIGHT) {
    int idx = screen_y * WIDTH + screen_x;
    if (seperZ > zBuffer[idx]) {
        zBuffer[idx] = seperZ;

        // Jika L > 0, permukaan menghadap cahaya
        if (L > 0) {
            // Mapping nilai L ke urutan karakter ASCII (paling gelap ke paling terang)
            const char* brightness = ".,--~:;=!*#$@";
            int b_idx = L * 8; // Menyesuaikan dengan panjang string
            Buffer[idx] = brightness[b_idx];
        } else {
            // Sisi yang membelakangi cahaya tetap diberikan karakter redup
            Buffer[idx] = '.';
        }
    }
}
}
```

Kode Akhir main()

```
int main(){
    // Bersihkan layar terminal
    printf("\x1b[2J");

    while(1){
        // Persiapkan fram
        // Reset screen buffer menjadi background ASCII (' ')
        memset(Buffer, bgASCII, WIDTH * HEIGHT);
        // Reset Z-Buffer
        memset(zBuffer, 0, WIDTH * HEIGHT * sizeof(float));

        // Panggil inis()
        inis();

        const int S = 20; // setengah panjang sisi kubus
        const float STEP = 0.5; // kepadatan titik per permukaan

        // Menghasilkan semua sisi kubus
        for(float kubuX = -S; kubuX<S; kubuX += STEP){
            for(float kubuY = -S; kubuY<S; kubuY += STEP){

                // Sisi depan diisi dengan @
                titikPemrosesan(kubuX, kubuY, S, 0, 0, 1);

                // Sisi belakang diisi dengan $
                titikPemrosesan(kubuX, kubuY, -S, 0, 0, -1);

                // Sisi kiri diisi dengan ~
                titikPemrosesan(-S, kubuY, kubuX, -1, 0, 0);

                // Sisi kanan diisi dengan #
                titikPemrosesan(S, kubuY, kubuX, 1, 0, 0);

                // Sisi atas diisi dengan ;
                titikPemrosesan(kubuX, -S, kubuY, 0, -1, 0);

                // Sisi bawah diisi dengan +
                titikPemrosesan(kubuX, S, kubuY, 0, 1, 0);
            }
        }

        // Kembalikan cursor terminal ke pojok kiri atas.
        printf("\x1b[H");

        // Loop Screen Buffer 1D dan print
        for(int k=0; k < WIDTH * HEIGHT; k++){
            // Terminal tidak tahu kapan baris array selesai
            // jika index sudah melebihi ukuran terminal, print '\n'
            // Selainnya, print karakter yang kita simpan di buffer
        }
    }
}
```

```

    putchar(k % WIDTH ? Buffer[k] : '\n');
}

// Increment sudutnya.
A += 0.6;    // rotasi sumbu X
B += 0.3;    // rotasi sumbu Y
C += 0.1;    // rotasi sumbu Z

usleep(16000); // Kecepatan sekitar 60 fps
}
return 0;
}

```